Лабораторная работа №2

«Метод ломаных»

Выполнил: Рожновский Юрий, п. 1.2

Объяснение работы программы

Весь код можно найти в файле polygonal_method.py.

Код разбит на несколько функций. Разберем их.

```
def reading():
    expression = input("Введите функцию: f(x) = ")
    a, b = map(float, input("Введите координаты ограничения a и b через пробел:
").split())
    ext = input("Введите что найти, min или max: ")
    eps = float(input("Введите точность вычисления: "))

    if a >= b:
        raise ValueError("Должно выполняться a < b.")
    if ext not in ("min", "max"):
        raise ValueError("Тип экстремума должен совпадать с min или max.")
    if eps <= 0:
        raise ValueError("Точность eps должна быть положительной.")

    return expression, a, b, ext, eps
```

Эта функция просто считывает начальные данные для работы и проверяет их на валидность.

```
def take_function(expression, ext):
    f = lambda x: eval(f"-({expression})" if ext == "max" else expression,
{**math.__dict__, "x": x})
    return f
```

В эту функцию передается выражение функции и тип экстремума, который мы ищем. Возвращает он функцию, которая принимает аргумент x и возвращает его значение для переданного выражения. Также для универсализации последующего решения, если мы ищем максимум, то выражение домножается на -1, чтобы можно было методом ломаных искать только минимум. Реализовано это через лямбда функцию с передачей как аргументов функций, доступных в библиотеке math.

```
def find_L(f, a, b, eps, num_points=None):
    if num_points is None:
        num_points = abs(int((b - a) / eps)) * 10

    x = np.linspace(a, b, num_points)
    y = np.array([f(xi) for xi in x])
    dx = np.diff(x)
```

```
    dy = np.abs(np.diff(y))
    L = np.max(dy / dx)

    return L * 1.2
```

Эта функция используется для нахождения константы Липшица (L). Она разбивает отрезок на множество отрезков, количество которых зависит от длины изначального отрезка и требуемой точности. После этого вычисляются значения на границах подотрезков и берется максимальное значения наклона прямой, проходящей через соседние точки. Дополнительно полученное значение домножается на 1.2 для избегания погрешностей.

```
def find_crossing_points(points, L, ext="min"):
    crossing_points = []

    for (x1, y1), (x2, y2) in zip(points, points[1:]):
        x_cross = (x1 + x2) / 2 + (y1 - y2) / (2 * L)
        x_cross = max(x1, min(x_cross, x2))
        y_cross = y1 + (1 if ext == "max" else -1) * L * abs(x_cross - x1)
        crossing_points.append((x_cross, y_cross))

    return crossing_points
```

Эта функция принимает точки на графике функции, константу L и при необходимости тип вычисляемого экстремума. Дальше мы разбиваем их на соседствующие пары и для каждой пары по формуле находим x для точки пересечения ломаной, при необходимости возвращаем x в границы отрезка и по формуле находим y точки пересечения. Добавляем найденную точку в массив и потом возвращаем весь массив.

```
def piyavskii(f, a, b, L, eps):
    start_time = time.perf_counter()
    iterations = 0

    points = [(a, f(a)), (b, f(b))]

    while True:
        points.sort(key=lambda p: p[0])
        best_y = min(y for x, y in points)
        low_points = find_crossing_points(points, L, ext="min")

        x_new, lower_min = min(low_points, key=lambda p: p[1])
```

```
        if best_y - lower_min < eps:
            break

        y_new = f(x_new)
        points.append((x_new, y_new))
        iterations += 1

    elapsed_time = time.perf_counter() - start_time

    return points, iterations, elapsed_time
```

Это функция выполнения непосредственно метода Пиявского. Для статистики мы замеряем время работы и количество выполненных операций. Основной принцип работы: мы имеем массив точек пересечения ломаной и данной функции. На каждой итерации находим точку ломаной с наименьшим значением и по полученному x добавляем новую точку в массив точек графика функции. Причиной остановки выступает ситуация, когда разница наименьших значений функции и ломаной меньше заданного эпсилон. Возвращается все полученные точки функции, кол-во итераций и время работы метода.

```
def calculation(expression, a, b, ext, eps):
    f = take_function(expression, ext)
    L = find_L(f, a, b, eps)

    points, iterations, elapsed_time = piyavskii(f, a, b, L, eps)

    if ext == "max":
        points = [(x, -y) for x, y in points]
        f = take_function(expression)

    print_stats(L, points, iterations, elapsed_time)
    visualize(f, a, b, ext, L, eps, points)
```

Это функция, объединяющая все выполнение. По заданным данным находятся функция (под поиск минимума) и коэффициент L, применяется метод Пиявского, если мы изначально перевернули функцию, переворачиваем снова, в том числе найденные точки, после чего выводим статистику и визуализируем графики.

Код для статистики и визуализации разбирать не вижу особого смысла.

Примеры работы

Пример 1

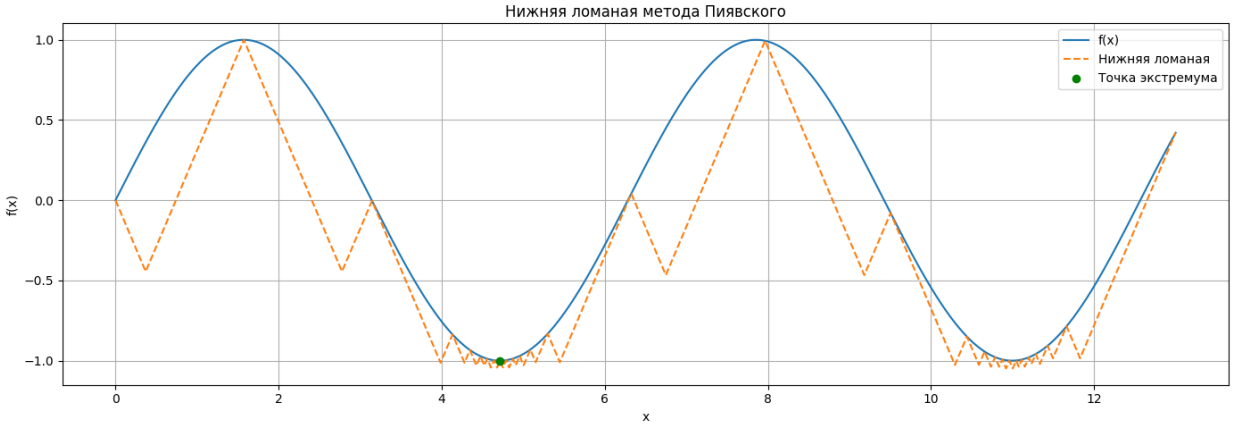
Пусть начальные данные следующие:

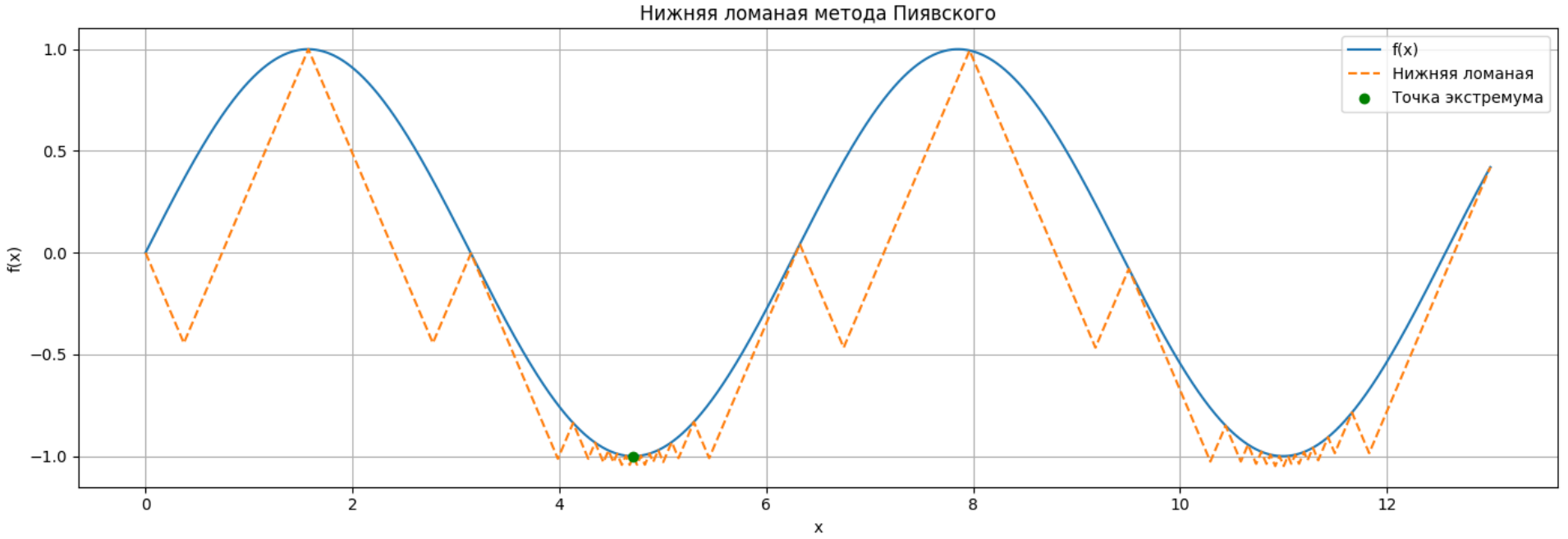
```
expression = "sin(x)"
a, b = 0, 13
ext = "min"
eps = 0.05
```

Получим статистику:

```
Результаты работы метода Пиявского
-----------------------------------
Коэффициент Липшица:   L = 1.199998
Точка экстремума:      x = 4.716163
Значение функции:      f(x) = -0.999993
Количество итераций:   27
Время работы:          0.000475 сек.
```

И графики:




Нижняя ломаная метода Пиявского
f(x)
Нижняя ломаная
Точка экстремума
1.0
0.5
0.0
−0.5
−1.0
f(x)
0
2
4
6
8
10
12
x

Пример 2

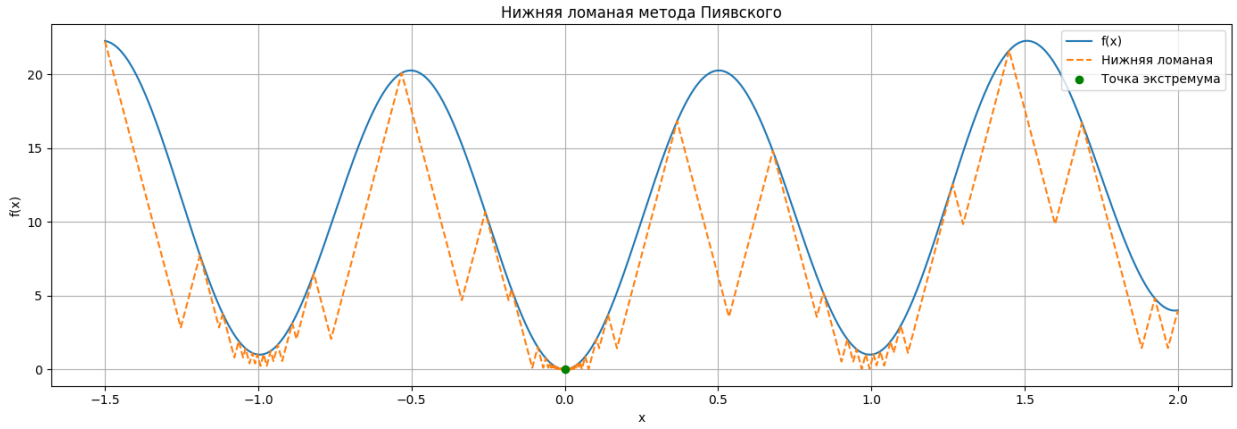
Рассмотрим функцию Растригина со следующими параметрами:

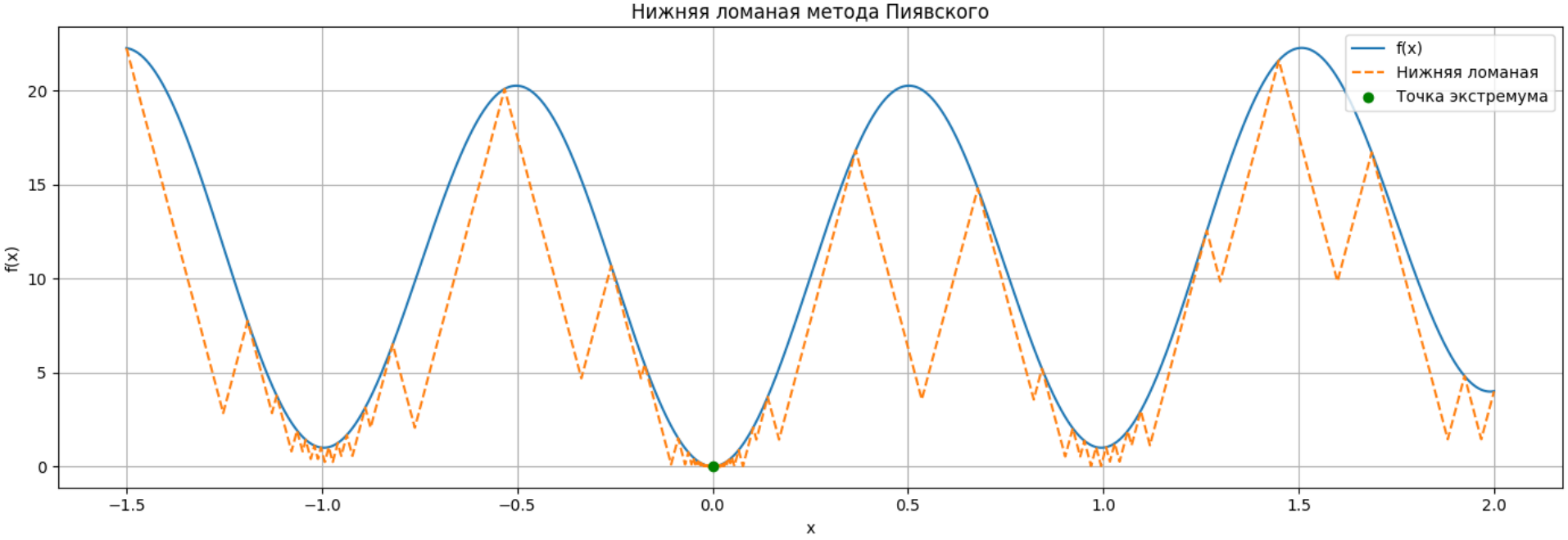
```
expression = "10 + x**2 - 10 * cos(2 * pi * x)"
a, b = -1.5, 2
ext = "min"
eps = 0.01
```

Статистика:

```
Результаты работы метода Пиявского
-----------------------------------
Коэффициент Липшица:   L = 78.398855
Точка экстремума:      x = 0.000057
Значение функции:      f(x) = 0.000001
Количество итераций:   153
Время работы:          0.010274 сек.
```

И графики:




Нижняя ломаная метода Пиявского
f(x)
Нижняя ломаная
Точка экстремума
20
15
10
5
0
f(x)
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
x

Вывод

Был изучен способ поиска экстремумов на отрезке функции при помощи метода Пиявского. Также была написана программа для применения метода автоматически.